

Advanced JavaScript answer key

Alternative solutions are acceptable when they produce the required behaviour and use the concepts requested in the question.

Section A: DOM and browser interaction

Exercise 1: Update the page after a button click

script.js

```
const bookingStatus =
  document.getElementById(
    "booking-status"
  );

const confirmButton =
  document.querySelector(
    "#confirm-booking"
  );

function handleBookingConfirmation() {
  bookingStatus.textContent =
    "Booking confirmed";
}

confirmButton.addEventListener(
  "click",
  handleBookingConfirmation
);
```

`handleBookingConfirmation` is passed as a function reference. The browser calls it after the button receives a click.

Writing `handleBookingConfirmation()` in the listener would call the function while the listener is being registered.

Exercise 2: Compare `event.target` and `event.currentTarget`

When the user clicks the text inside the `<span>`, the original code displays:

SPAN / BUTTON

- `event.target` is the element where the event began.

- `event.currentTarget` is the element that owns the current listener.

Corrected handler:

```
deliveryButton.addEventListener(  
  "click",  
  event => {  
    const action =  
      event.currentTarget.dataset.action;  
  
    eventResult.textContent =  
      `Action: ${action}`;  
  }  
);
```

Expected result:

Action: dispatch

Using `currentTarget` makes the result consistent whether the user clicks the `<span>` or another part of the button.

Exercise 3: Add and remove notices

script.js

```
const noticeStatus =  
  document.querySelector(  
    "#notice-status"  
  );  
  
const addNoticeButton =  
  document.querySelector(  
    "#add-notice"  
  );  
  
const removeNoticeButton =  
  document.querySelector(  
    "#remove-notice"  
  );  
  
const noticelist =  
  document.querySelector(  
    "#notice-list"  
  );  
  
let nextNoticeNumber = 1;
```

```

function handleAddNotice() {
  const noticeItem =
    document.createElement("li");

  noticeItem.textContent =
    `Notice ${nextNoticeNumber}`;

  noticeList.append(noticeItem);

  nextNoticeNumber += 1;

  noticeStatus.textContent =
    "Notice added";
}

function handleRemoveNotice() {
  const latestNotice =
    noticeList.querySelector(
      "li:last-child"
    );

  if (!latestNotice) {
    noticeStatus.textContent =
      "No notices to remove";
    return;
  }

  latestNotice.remove();

  noticeStatus.textContent =
    "Latest notice removed";
}

addNoticeButton.addEventListener(
  "click",
  handleAddNotice
);

removeNoticeButton.addEventListener(
  "click",
  handleRemoveNotice
);

```

Creating the `<li>` does not make it visible. `append` inserts it into the list.

The empty-list check prevents `.remove()` from being called on `null`.

Exercise 4: Choose between `textContent` and `innerHTML`

`script.js`

```
const messageInput =
  document.querySelector(
    "#message-input"
  );

const showMessageButton =
  document.querySelector(
    "#show-message"
  );

const messagePreview =
  document.querySelector(
    "#message-preview"
  );

const trustedSummary =
  document.querySelector(
    "#trusted-summary"
  );

function handleShowMessage() {
  messagePreview.textContent =
    messageInput.value;
}

showMessageButton.addEventListener(
  "click",
  handleShowMessage
);

trustedSummary.innerHTML = `
  <h2>Registration complete</h2>
  <p>Your seat has been reserved.</p>
`;
```

`textContent` is used for the input because the value should appear as text. The browser does not interpret the supplied tags as HTML.

`innerHTML` is used for the prepared summary because the application intentionally supplies trusted markup that should create elements.

Exercise 5: Display Browser Object Model information

script.js

```
const showDetailsButton =
  document.querySelector(
    "#show-details"
  );

const pageAddress =
  document.querySelector(
    "#page-address"
  );

const browserLanguage =
  document.querySelector(
    "#browser-language"
  );

const screenWidth =
  document.querySelector(
    "#screen-width"
  );

function handleShowDetails() {
  pageAddress.textContent =
    location.href;

  browserLanguage.textContent =
    navigator.language;

  screenWidth.textContent =
    `${screen.width}px`;
}

showDetailsButton.addEventListener(
  "click",
  handleShowDetails
);
```

DOM work:

- Selecting the button and `<span>` elements
- Registering the event listener
- Updating `textContent`

Browser Object Model work:

- Reading `location.href`
- Reading `navigator.language`

Section B: Modern JavaScript syntax

Exercise 6: Extract values with destructuring

```
const booking = {
  bookingId: "BK-204",
  guestName: "Ira Sen",
  status: "Confirmed",
  preferences: [
    "Window seat",
    "Vegetarian meal",
    "Email updates"
  ]
};

const {
  bookingId,
  guestName,
  status
} = booking;

const [
  primaryPreference,
  ...otherPreferences
] = booking.preferences;

console.log(
  `${bookingId} | `
  + `${guestName} | `
  + `${status}`
);

console.log(
  `Primary preference: `
  + `${primaryPreference}`
);

console.log(
  `Other preferences: `
  + `${otherPreferences.length}`
);
```

Expected output:

BK-204 | Ira Sen | Confirmed
Primary preference: Window seat
Other preferences: 2

Object destructuring matches property names. Array destructuring reads values by position.

The rest element collects the remaining array values.

Exercise 7: Update data without changing the original

```
const product = {
  id: "PR-108",
  name: "Wireless mouse",
  stock: 4,
  tags: [
    "electronics",
    "accessories"
  ]
};

const updatedProduct = {
  ...product,
  stock: 7,
  tags: [
    ...product.tags,
    "featured"
  ]
};

console.log(product.stock);
console.log(product.tags);

console.log(updatedProduct.stock);
console.log(updatedProduct.tags);
```

Expected results:

```
4
["electronics", "accessories"]
7
["electronics", "accessories", "featured"]
```

Object spread creates a new outer object. It does not automatically create a new nested array.

If `updatedProduct` continued using the original `tags` array and the array were mutated, both objects would refer to the same array. Array spread creates a separate tags array before adding the new value.

Exercise 8: Collect extra arguments with a rest parameter

```
function createActivitySummary(
  title,
  ...learners
) {
  return (
    `${title}: `
    + `${learners.length} `
    + `${
      learners.length === 1
        ? "learner"
        : "learners"
    }\\n`
    + learners.join(", ")
  );
}

console.log(
  createActivitySummary(
    "DOM practice",
    "Aman",
    "Jaya",
    "Omar"
  )
);

console.log(
  createActivitySummary(
    "Promise review",
    "Leena"
  )
);
```

Expected output:

```
DOM practice: 3 learners
Aman, Jaya, Omar

Promise review: 1 learner
Leena
```

`...learners` collects every remaining argument into an array.

Exercise 9: Preserve `this` inside a delayed callback

```
const reminder = {
  message: "Submit the assignment",

  start() {
    setTimeout(() => {
      console.log(this.message);
    }, 500);
  }
};

reminder.start();
```

Expected output after the delay:

```
Submit the assignment
```

When `start()` is called as `reminder.start()`, `this` inside the method refers to `reminder`.

The arrow callback does not create a separate `this`. It uses the value from the surrounding `start` method.

Changing `start` into an arrow function would remove the method-style `this` binding that the solution needs.

Exercise 10: Refactor a formatter

```
const formatSession = session => {
  const {
    title,
    startTime,
    room
  } = session;

  return (
    `${title} starts at `
    + `${startTime} in ${room}`
  );
};

const session = {
  title: "JavaScript modules",
  startTime: "10:00",
  room: "Lab 3"
```

```
};

console.log(
  formatSession(session)
);
```

Expected output:

```
JavaScript modules starts at 10:00 in Lab 3
```

A one-expression version is also valid:

```
const formatSession = ({
  title,
  startTime,
  room
}) =>
  `${title} starts at `
  + `${startTime} in ${room}`;
```

Section C: JavaScript modules

Exercise 11: Repair a browser module

The corrected file structure remains:

```
exercise-11/
├─ index.html
├─ app.js
└─ pricing.js
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
  <title>Order total</title>
</head>
<body>
  <p id="order-total"></p>
```

```
<script
  type="module"
  src="./app.js"
></script>
</body>
</html>
```

pricing.js

```
export function calculateSubtotal(
  price,
  quantity
) {
  return price * quantity;
}
```

app.js

```
import {
  calculateSubtotal
} from "./pricing.js";

const orderTotal =
  document.querySelector(
    "#order-total"
  );

orderTotal.textContent =
  calculateSubtotal(250, 3);
```

Corrections:

1. The script uses `type="module"`.
2. The named import matches `calculateSubtotal`.
3. The relative path begins with `./`.
4. The browser module path includes `.js`.
5. The function call uses the imported name.

Expected page content:

750

Exercise 12: Split one script by responsibility

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
  <title>Selected course</title>
</head>
<body>
  <h1 id="course-title"></h1>

  <script
    type="module"
    src="./app.js"
  ></script>
</body>
</html>
```

course-data.js

```
export const courses = [
  {
    id: 1,
    title: "JavaScript fundamentals"
  },
  {
    id: 2,
    title: "Advanced JavaScript"
  }
];
```

course-utils.js

```
export function findCourse(
  courseList,
  courseId
) {
  return courseList.find(
    course => course.id === courseId
  );
}
```

app.js

```
import {
  courses
} from "../course-data.js";

import {
  findCourse
} from "../course-utils.js";

const courseTitle =
  document.querySelector(
    "#course-title"
  );

const selectedCourse =
  findCourse(courses, 2);

courseTitle.textContent =
  selectedCourse.title;
```

Expected page content:

Advanced JavaScript

The data module stores the data. The utility module contains reusable search logic. The entry module connects those pieces to the page.

Exercise 13: Import only the public functions

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
  <title>Formatting</title>
</head>
<body>
  <script
    type="module"
    src="../app.js"
  ></script>
```

```
</body>
</html>
```

app.js

```
import {
  formatName,
  formatStatus
} from "./formatters.js";

console.log(
  formatName("meera")
);

console.log(
  formatStatus("confirmed")
);
```

Expected output:

```
Meera
Status: Confirmed
```

Answers:

1. `formatName` and `formatStatus` include `export`, so other modules can import them.
2. `capitalize` is not exported. It remains internal to `formatters.js`.
3. A named import requests a specific exported binding. The requested name must match the exported name.

Section D: Asynchronous JavaScript

Exercise 14: Predict the execution order

The messages appear in this order:

```
Start
End
Timer finished
```

`setTimeout` schedules the callback to run later. JavaScript continues to the next available statement and prints `End`.

A delay of `0` means there is no requested waiting period beyond scheduling. It does not interrupt the currently running code.

Exercise 15: Pass a callback instead of calling it immediately

```
function showStatus(message) {  
  console.log(message);  
}  
  
setTimeout(() => {  
  showStatus("Ready");  
}, 500);
```

Expected output after the delay:

Ready

`showStatus` is a reference to the function.

`showStatus()` calls the function immediately.

An arrow callback is passed to `setTimeout`. The browser calls that callback later, and the callback then calls `showStatus` with the required argument.

Exercise 16: Report success or failure through callbacks

```
function loadSeatAvailability(  
  shouldFail,  
  onSuccess,  
  onError  
) {  
  setTimeout(() => {  
    if (shouldFail) {  
      onError(  
        "Seat information "  
        + "could not be loaded"  
      );  
      return;  
    }  
  
    onSuccess({  
      availableSeats: 6  
    });  
  }, 500);  
}  
  
function handleSeatSuccess(result) {  
  console.log(  

```

```

        `Available seats: `
        + `${result.availableSeats}`
    );
}

function handleSeatError(message) {
    console.log(message);
}

loadSeatAvailability(
    false,
    handleSeatSuccess,
    handleSeatError
);

```

Success output:

```
Available seats: 6
```

Failure test:

```

loadSeatAvailability(
    true,
    handleSeatSuccess,
    handleSeatError
);

```

Failure output:

```
Seat information could not be loaded
```

The loader decides when the operation finishes. The supplied callbacks decide what the caller does with the result.

Exercise 17: Trace dependent callbacks

Answers:

1. `loadOrder` starts first.
2. `order.customerId` does not exist for the caller until `loadOrder` finishes and supplies the order to its callback.
3. The printed message is:

Nikhil: 850

1. More dependent operations would add more nesting and make the sequence, variables, and error handling harder to follow.

One valid refactoring is:

```
function showOrderSummary(
  order,
  customer
) {
  console.log(
    `${customer.name}: `
    + `${order.total}`
  );
}

function handleOrderLoaded(order) {
  loadCustomer(
    order.customerId,
    customer => {
      showOrderSummary(
        order,
        customer
      );
    }
  );
}

loadOrder(
  "OR-108",
  handleOrderLoaded
);
```

The dependency remains the same. The customer request starts only after the order supplies the customer ID.

Exercise 18: Create and consume a Promise

```
function checkRoomAvailability(
  roomNumber,
  shouldFail
) {
  return new Promise(
    (resolve, reject) => {
      setTimeout(() => {
        if (shouldFail) {
```

```

        reject(
            new Error(
                "Room information "
                + "could not be loaded"
            )
        );
        return;
    }

    resolve({
        roomNumber,
        available: true
    });
}, 500);
}
);
}

```

Success test:

```

checkRoomAvailability(
    204,
    false
)
.then(room => {
    if (room.available) {
        console.log(
            `Room ${room.roomNumber} `
            + "is available"
        );
    }
})
.catch(error => {
    console.log(error.message);
});

```

Expected success output:

```
Room 204 is available
```

Failure test:

```

checkRoomAvailability(
    204,
    true
)
.then(room => {
    console.log(room);

```

```
    })  
    .catch(error => {  
      console.log(error.message);  
    });
```

Expected failure output:

```
Room information could not be loaded
```

`resolve` supplies a fulfilled value to `.then()`. `reject` supplies the failure to `.catch()`.

Exercise 19: Return a value to the next Promise stage

```
loadProfile()  
  .then(profile => {  
    const updatedProfile = {  
      ...profile,  
      theme: "Dark"  
    };  
  
    return updatedProfile;  
  })  
  .then(updatedProfile => {  
    console.log(  
      updatedProfile.theme  
    );  
  })  
  .catch(error => {  
    console.log(error.message);  
  });
```

Expected successful output:

```
Dark
```

Answers:

1. The original callback has a block body with no `return`, so it returns `undefined`.
2. A `.then()` callback's return value becomes the fulfilled value received by the next `.then()`.
3. An error thrown in either callback moves to the final `.catch()`.

The callback can also return the object directly:

```
loadProfile()  
  .then(profile => ({
```

```
    ...profile,
    theme: "Dark"
  )))
  .then(updatedProfile => {
    console.log(
      updatedProfile.theme
    );
  })
  .catch(error => {
    console.log(error.message);
  });
```

Exercise 20: Rewrite a Promise chain with `async` and `await`

```
async function showBooking(
  shouldFail
) {
  try {
    const booking =
      await loadBooking(shouldFail);

    const {
      bookingId,
      status
    } = booking;

    console.log(
      `${bookingId}: ${status}`
    );
  } catch (error) {
    console.log(error.message);
  }
}

showBooking(false);
```

Expected success output:

```
BK-501: Confirmed
```

Failure test:

```
showBooking(true);
```

Expected failure output:

Booking could not be loaded

Answers:

1. An `async` function always returns a Promise.
2. `await` provides the Promise's fulfilled value inside the asynchronous function.
3. When the awaited Promise rejects, execution moves to the matching `catch` block.

Test the success and failure calls separately so their delayed output remains easy to follow.

Final exercise: Build a workshop availability viewer

`workshop-service.js`

```
export function loadWorkshop(
  shouldFail
) {
  return new Promise(
    (resolve, reject) => {
      setTimeout(() => {
        if (shouldFail) {
          reject(
            new Error(
              "Workshop could "
              + "not be loaded"
            )
          );
          return;
        }
        resolve({
          id: "WS-208",
          title:
            "JavaScript practice lab",
          facilitator: "Neha",
          status: "Scheduled",
          topics: [
            "DOM",
            "Modules",
            "Promises"
          ]
        });
      }, 600);
    }
  );
}
```

```
    );  
  }  
}
```

ui.js

```
const workshopStatus =  
  document.querySelector(  
    "#workshop-status"  
  );  
  
const workshopDetails =  
  document.querySelector(  
    "#workshop-details"  
  );  
  
export function showStatus(  
  message  
) {  
  workshopStatus.textContent =  
    message;  
}  
  
export function clearWorkshop() {  
  workshopDetails.textContent = "";  
}  
  
export function showWorkshop(  
  workshop  
) {  
  const {  
    id,  
    title,  
    facilitator,  
    topics  
  } = workshop;  
  
  const openWorkshop = {  
    ...workshop,  
    status: "Open"  
  };  
  
  workshopDetails.innerHTML = `  
    <h2>${id}</h2>  
    <p>${title}</p>  
    <p>  
      Facilitator: ${facilitator}  
    </p>  
    <p>  
      Status: ${openWorkshop.status}  
    </p>
```

```

    <p>
      Topics: ${topics.join(", ")}
    </p>
  `;
}

```

A version that creates each element with `createElement` and `textContent` is also valid.

The markup here is created from prepared application data. No user-entered HTML is inserted.

main.js

```

import {
  loadWorkshop
} from "../workshop-service.js";

import {
  showStatus,
  showWorkshop,
  clearWorkshop
} from "../ui.js";

const loadWorkshopButton =
  document.querySelector(
    "#load-workshop"
  );

const simulateErrorButton =
  document.querySelector(
    "#simulate-error"
  );

async function handleWorkshopLoad(
  shouldFail
) {
  clearWorkshop();

  showStatus(
    "Loading workshop..."
  );

  try {
    const workshop =
      await loadWorkshop(shouldFail);

    showWorkshop(workshop);

    showStatus(
      "Workshop loaded"
    );
  }
}

```

```

    );
  } catch (error) {
    clearWorkshop();

    showStatus(
      error.message
    );
  }
}

loadWorkshopButton.addEventListener(
  "click",
  () => {
    handleWorkshopLoad(false);
  }
);

simulateErrorButton.addEventListener(
  "click",
  () => {
    handleWorkshopLoad(true);
  }
);

```

Expected successful state

Status:

Workshop loaded

Details:

WS-208
 JavaScript practice lab
 Facilitator: Neha
 Status: Open
 Topics: DOM, Modules, Promises

Expected failure state

Workshop could not be loaded

The previous workshop details are removed before the failure message is displayed.

Concepts used

- DOM selection and event listeners
- Named exports and imports
- Relative module paths
- A Promise-based asynchronous service
- An `async` function
- `await`
- `try` and `catch`
- Object destructuring
- Object spread
- Array `join`
- Visible loading, success, and error states